

IssueForge v1 decomposition, attempt 2 — FAILURE REPORT

(no issues created)

Status: BLOCKED. The independent review gate rejected the decomposition twice. Per the run protocol, **no GitHub issues were created and no resolution was invented.** This report records what was fixed, what remains, and the **two decisions** needed to unblock.

Date	2026-07-14
Repository	MatthewDruhl/IssueForge (private)
Source PRD	#1 — amended with D1–D4
PRD scale	59 acceptance criteria across 11 user stories (was 51 before the D1–D4 amendment)
Draft	25 v1 child issues + 2 deferred–v2 (docs/planning/decomposition-draft-v3.md)
Review 1 (draft v3)	fresh session → REVISE (8 blocking findings)
Review 2 (draft v3, repaired)	fresh session → REVISE (5 blocking findings; 3 of 8 confirmed FIXED)
Child issues created	0
Issues modified	0 (PRD #1 untouched)
MARVIN files/issues modified	0

Executive summary

The gate held, and it was right to. But this attempt is **not** a repeat of the last one: the failure moved from *"the plan conflicts with the PRD"* to *"the plan's mechanisms are not yet airtight."* That is progress, and it is measurable.

What the amendment bought. The prior attempt was blocked by three PRD conflicts (D1 pytest-only, D2 the review override, D3 shaper ordering). All three are now settled PRD text, and **not one of them resurfaced in either review round.** The amendment worked.

What this attempt found on its own, before any reviewer saw it. The verification pass caught the prior draft shipping **eight false claims about MARVIN's source** into what would have become GitHub issues — most importantly a **false "nothing to port"** on five canonical artifacts that *do* exist (§4). It also re-derived the criterion count: **the PRD has 59 criteria, not 51**, and the old matrix would have **silently dropped all eight** the amendment added.

Why it is still blocked. Five findings survived the second round. **Three are mechanical** — the draft says the right thing and simply fails to write it in the right places; they need no decision and are specified in §3. **Two require design decisions that are the author's call, not the decomposer's** (§2). Inventing them is exactly what the protocol forbids, and exactly what produced the one real error of the last run.

1. The two decisions needed to unblock

D5 — How is a file's ROLE determined: contract, implementation, or both? (*blocking*)

This is the sharpest defect found in either round, and the decomposition cannot proceed without an answer.

Draft v3 tried to derive a file's role from **approved-scope membership**: *a path inside the approved file scope is the system under test and is expected to change; a path outside it that feeds test collection or outcome is protected; a path in both is a contradiction and fails the freeze.*

That rule cannot hold, because the acceptance tests are in the PR. prd-v1.md:88 (US-7): "one reviewable PR containing **the approved tests and implementation**." And prd-v1.md:161: "One branch contains a separately committed approved test contract **followed by implementation**." So the test files and their fixtures are **necessarily part of the delivered diff** — and therefore must be inside the approved file scope that US-6.4 checks at readiness. They are **also**, necessarily, **protected contract inputs** (US-5.5, US-5.6).

Every branch of the rule breaks:

- **Exclude the tests from the approved scope** → S15's scope check fails on the very PR that is supposed to contain them.
- **Put them in both sets** → the draft's own rule calls that a contradiction and **refuses to freeze**.
- **Drop the discriminator** → S13's absolute protected-path gate freezes the implementation too, and **the issue becomes unbuildable** (round 1, finding 7).

The fix is to model file roles explicitly rather than infer them, but the *policy* is a real decision:

- **(a) Three explicit roles in the buildability contract.** Every path in the approved scope carries a role: `contract` (immutable after freeze), `implementation` (mutable), or — if the intersection is permitted — `contract` wins after freeze. The approved scope may legitimately contain both; **their intersection is governed by the stricter contract rule**. This is the reviewer's recommendation and the smallest change.
- **(b) Two disjoint scopes.** The buildability contract approves an `implementation_scope` and the acceptance contract separately freezes a `contract_scope`; the readiness check unions them for the diff and applies the contract rule to the contract half. Cleaner conceptually, but it means the human approves scope **twice**, at two different gates.
- **(c) Timeline-based.** A contract path is mutable **before** the freeze commit and immutable **after** it, with no role labels at all. Simplest, but it makes "which files may the implementer touch?" unanswerable until the freeze exists — and US-3.6 requires the human to approve the file scope **before** contract authoring begins.

Recommendation: (a). It preserves the single US-3.6 approval gate, keeps the PRD's "approved scope carried forward into the freeze" (US-5.5) intact, and makes the intersection a *governed, normal* case instead of an error. **But it is a PRD-adjacent policy call and needs your sign-off**, because it decides what a human is actually approving at the US-3.6 gate.

Related, and settled by the same decision: S13 currently recomputes **every frozen dependency hash** at the candidate but **never re-resolves the external package identities and versions that S12 froze**. A plugin version swap changes the contract's meaning with every file hash intact. Whatever D5 decides, S13 must **re-probe and compare external identities/versions in the authoritative verification environment**.

D6 — What is the unit of a "behavior" in the source audit? (*blocking*)

prd-v1.md:132 (US-11.1) requires a design record that inventories "*the corresponding MARVIN skills, scripts, **tests**, and relevant **failure-driven updates***." :133 (US-11.2) requires **every** inventoried behavior classified **with a reason**.

Draft v3 made the inventory unit a *behavior* and added a lint failure mode for "*a discovered behavior inside an inventoried artifact with no disposition*." **But it then defined discovery as enumerating public symbols, associated test files, and commit/issue references found inside the file.** The reviewer's objection is correct and concrete:

- **A single public symbol can implement several independently meaningful safeguards.** `merged_runner.py` is the proof: it carries **six-plus distinct failed-read inversions**, each one a separate shipped-bug lesson. A record naming the file and classifying one of them would pass the lint.
- **Private symbols can be load-bearing** (`_reachability`, `_remote_branch_present` — both private, both critical).
- **A relevant failure-driven update need not be referenced inside the source or its tests.**

So failure mode five cannot fire on exactly the omissions it exists to catch.

The decision: what makes a behavior *discoverable and countable*?

- **(a) Behavior = a named, checked-in claim.** The audit author writes behavior records; discovery's job is to find **artifacts and tests**, and a human approves that the behavior list is complete. Honest about the limit: a lint cannot enumerate semantics. **Cheapest; leans on the human gate.**
- **(b) Behavior = a test.** Every MARVIN test is a behavior; the inventory requires a disposition for **every test in the discovered artifact's test files**. Mechanically enumerable and directly serves US-11.4 ("tests explaining reused safeguards are ported"). Misses behaviors with no test.
- **(c) Behavior = a branch/guard.** AST-enumerate every early-return, raise, and tri-state guard in the artifact. This *would* catch all six inversions. Highest fidelity, most noise, most build cost.

Recommendation: (b), with (a)'s human approval on top. A test is a real, countable, checked-in artifact; it is the unit US-11.4 already speaks in; and it makes "did you port the test that explains this safeguard?" mechanically checkable. The human approval covers what tests cannot.

2. The five blocking findings that remain

#	Issue	Finding	Needs
1	S2	Behavior completeness still reduces a behavior to a discoverable symbol or reference. Failure mode five cannot fire on private behaviors, multi-safeguard symbols, or unreferenced updates.	D6
2	S4 / S7 / S10 / S11 / S15	S4 <i>promises</i> that "each later producer carries its own canary assertion," but the producer issues do not contain those acceptance criteria . S7 says only "full output persists through S4's redacting writer"; S11 says "the review packet persists, redacted." Those are statements, not tests.	mechanical
3	S8 / S15	The two deterministic analyses were added, but the old contradictory language was left in place : "a heuristic feeding a judgment call," "one <code>classify(diff)</code> used as a HINT." And S15 asserts required events / logger conventions / sensitive-field exclusion are "deterministically verified" without defining the evidence .	mechanical + one spec
4	S12 / S13	The closure discriminator breaks on the acceptance tests themselves (see D5). Separately, S13 recomputes file hashes but never re-resolves the frozen external package identities/versions .	D5
5	S25 / S3 / S4	S25 (the write seam) is <i>called</i> a gate for every disk-writing issue, but the issue table lists S4 as blocked only by S1 and S3 , and S3 persists the registry while S25 claims to gate only "S4 onward." The build graph therefore permits persistent writers to land before the seam that constrains them .	mechanical

The three mechanical fixes, specified and ready to apply once D5/D6 land:

1. Add explicit **canary acceptance criteria** to every producer of prompts, responses, command output, diffs, review packets, events, and error traces (S7, S10, S11, S15) — each exercising **success and failure** persistence paths through S4's API. Keep S24's lifecycle canary as the exhaustive backstop; it does not replace producer-local tests for error and timeout branches a single lifecycle may never hit.
2. **Delete S8's obsolete heuristic/hint language.** Specify `classify_prospective` and `classify_diff` as two separately testable APIs, and define what *evidences* each US-6.9 obligation (logger selection, level/format/ correlation reuse, sensitive-field non-leakage). Anything that remains irreducibly semantic goes to the AI reviewer **and is therefore overridable** — so it must be named, not left ambiguous.
3. Make **S25 a hard blocker of S3 and S4** in the issue table *and* in each issue body, and add a CI-order assertion that no writing module can land before the seam and lint are active.

3. What review 2 confirmed as FIXED

Recorded so the next attempt does not re-litigate settled ground.

- **The runtime graph no longer bypasses shaping.** `buildable` routes through S20 (approve + apply) and only then reaches S10; oversized routes to S21, whose children enter the queue while **the parent run stops**; blocked pauses. **S10 is unreachable until the applicable shaping mutation completes** — so US-3.1 can no longer be satisfied by an issue nobody revised.
- **The baseline-green mechanism is sound.** The unsound `canonical_collect(base) - new_acceptance_ids` subtraction is gone. `BASE_IDS` is snapshotted at base; **every** base id runs at the candidate; a disappeared **or reused** base id is a hard failure. *(The subtraction would have silently removed a preexisting test from the very check that protects it.)*
- **S11 no longer consumes a counter its own downstream producer owns.** Contract review has its own `contract_review_rounds`; any test change re-runs the S10 predicates and mints **new sha-bound red evidence** before S12 may freeze.
- **The override is correctly bounded** (US-6.5): human-only, per-finding, sha-bound, invalidated by any later change, preceded by one fresh replacement review, non-waiving of every deterministic gate, disclosed in the PR, and **not** merge authorization.
- **The push-order and closeout defects are materially corrected.** S14 never pushes; S16 is the only slice that pushes, and only after the gate passes (US-7.1). S18 keys closure on the **persisted, repository-qualified run-issue identity**.
- **Both deferred-v2 items remain visibly outside the v1 criterion graph.** Nothing was smuggled in.

4. Source-audit corrections — eight false claims caught before they became issues

Every prior-art claim in the superseded draft was re-read against the real file. **Eight were wrong.** Shipping any of them into a GitHub issue would have handed the gate a free kill, and several would have propagated into implementation.

Prior claim	Verdict
<code>_parse_pytest_summary</code> "cannot see pytest exit 5"	FALSE. <code>merged_runner.py:677</code> is <code>if failed > 0 or result.returncode != 0</code> — exit 5 is non-zero, so it is caught and does halt. It fails closed . The real defect is a mislabel (exit 5/2/3/4 all report as <code>red-main</code> with <code>passed: 0, failed: 0</code>).
"MARVIN has no workflow/state, no queue, no retention, no epic, no decomposition — nothing to port"	FIVE OF SEVEN ARE FALSE. Only the TUI is cleanly net-new. See below.
<code>check_build_pr_base.py</code> is ~100 lines	FALSE. 68 lines — and <code>default_branch</code> defaults to "main" , which the port must make required.
The golden-value arrow proxy is in <code>validate_accept_body.py</code>	FALSE. That file checks arrow presence only ; <code>... → ...</code> and <code>TBD → TBD</code> pass it. The both-sides non-placeholder proxy is in <code>validate_spec_up_issue.py</code> .
<code>check_acceptance_integrity.py</code> "excludes <code>conftest.py</code> from its guard"	Mis-framed. Its <code>:79–82</code> limit is a value-resolution scope limit, not a file-scope exclusion.
"five inversions" in <code>merged_runner.py</code>	Undercounted — there are six-plus.
<code>:828–836</code> is a "blanket halt-on-red-main"	Imprecise. It halts on any gate anomaly, and it is per-PR , not global.
The closeout chain includes a parent-epic update	FALSE. <code>grep -i "epic parent"</code> over <code>merged_runner.py</code> and <code>merged/SKILL.md</code> returns zero matches. US-8.2's epic update is new engine policy , not an extraction.

The five false "nothing to port" claims — real, tested MARVIN code the prior draft would have discarded:

`prune_plan_files.py` (a 30-day retention sweep **with an injectable clock**); `close_run_for_pr` + `VALID_STATUSES / TERMINAL_STATUSES` (a **guarded state transition** — `running` is never promoted straight to merged); `issues_to_findings.py` (footprint extraction that **forbids an empty footprint**); `schedule_waves.py` (a deterministic

conflict-detecting scheduler — not needed by single-run v1, but real); and `skills/prd-to-issues/SKILL.md` (the decomposition procedure itself).

This is not a footnote — it is the proof that review 02's source-audit finding was correct. A lint that checks a record against a curated "Initial source map" cannot establish completeness. Only discovery can.

5. Two live MARVIN defects found during verification

Found while verifying prior-art claims. **Neither was filed** — this run is forbidden from modifying MARVIN. Both are recommended for filing as MARVIN issues.

1. `merged_runner` reports a fully-skipped suite as **GREEN**. Exit 0 with a summary of "12 skipped" yields `passed=0, failed=0, returncode=0` → **no anomaly** → main is treated as green, and `_process_pr` proceeds to **delete branches, remove worktrees, and close issues** (:841–862). A suite whose module-level `pytest.skip` fired (missing env var, missing optional dependency) **verifies nothing and reports green**. This is a **false green in the merge gate**, distinct from the known exit-5 mislabel. **Exit 0 is not green**.
2. `_Closeout.close_issues` never checks `res.returncode` (:509–515). A transient `gh` failure yields `info = {}` → `refs = []` → `action: "clean"`, exit 0 — **after** the branch and worktree were already deleted. A failed read is silently reported as *"there were no issues to close."* It is the one unguarded failed read in a file whose whole discipline is that a failed read is never negative evidence.

6. Verification

- **No GitHub issues were created, edited, labeled, commented on, or closed** in `MatthewDruhl/IssueForge`. **PRD #1 is unmodified**.
- The `epic / v1 / deferred-v2 / phase:0–phase:5 / route:*` labels created by the **previous** attempt still exist and are **still unused**. They were built for the superseded ordering; **the next attempt should reuse or replace them deliberately, not inherit them by accident**.
- **No MARVIN file, state, skill, ledger, configuration, generated artifact, or GitHub issue was modified**. MARVIN was read-only provenance throughout.
- No IssueForge source code or tests were changed. No implementation branches or PRs were created.
- **The review gate ran under the guarded-launch contract it mandates:** stdin closed, stderr captured to a file (never `2>/dev/null`), a wall-clock timeout (`perl -e 'alarm N; exec @ARGV'` — **timeout(1) does not exist on macOS**), full output persisted, and empty-output-or-non-zero treated as FAILED. The reviewer has **no network**, so every input was materialized to local disk first.

PDF verification. `issueforge-v1-decomposition-report-v2.pdf` — **6 pages, 0 blank pages**, every page rendered and inspected. Asserted: **exactly one** `<h1>` (the render aborts otherwise), zero paragraphs beginning with a `#` immediately followed by a digit (the lazy- `<h1>` hazard that broke the last report), and a stated page count matching the actual one. One defect was caught on the first generation and fixed: a bold marker placed inside an inline code span rendered literally as asterisks. Tables, code spans, and rules render correctly; no clipped text.

7. The exact next command

Both decisions require amending PRD #1 (or an explicit written answer to each). Once D5 and D6 are settled:

```
/prd-to-issues for MatthewDruhl/IssueForge#1
```

Re-run against the amended PRD, **reusing** `docs/planning/decomposition-draft-v3.md` as the starting point — it is 25 issues away from complete, three fixes are purely mechanical (§2), and **59/59 criteria are already owned with a single**

owner each. Do not start from zero, and do not re-derive the source audit: **\$4's corrections were paid for.**